

GhostMind — Documento de Concepção (Pitch)

Projeto N3 — Sistema Inteligente com Agentes

Disciplina: Inteligência Artificial — Engenharia de Software

Integrantes: Artur Zanella, Gabriel Wendorff, Guilherme Freiburger e Lucas Eduardo Mueller

Data: 16/06/2026

Release Notes — v1.0

Nome e versão

GhostMind v1.0 — Simulador de perseguição e fuga com agentes inteligentes em labirinto (inspirado no Pac-Man), executado no navegador.

Resumo executivo

GhostMind é um simulador interativo, inspirado no Pac-Man, em que múltiplos agentes-fantasma autônomos caçam um agente Pac-Man num labirinto em grade. Cada fantasma planeja seu trajeto em tempo real com busca **A***, enquanto o Pac-Man — autônomo ou controlado pelo jogador — decide suas fugas com busca **Greedy Best-First**. Um classificador **Naive Bayes** avalia, a cada passo, o nível de ameaça do estado de jogo (seguro / risco / crítico) e ajusta a postura do Pac-Man autônomo entre coletar e fugir. O sistema resolve o problema de navegação e perseguição inteligente em ambiente dinâmico e demonstra, ao vivo, o comportamento e o desempenho de diferentes estratégias de busca informada e de decisão sob incerteza.

Principais funcionalidades

- Labirinto jogável com Pac-Man, pílulas, pílulas de poder e múltiplos fantasmas, em **modo Jogador** (controle manual por setas / WASD) e **modo IA** (Pac-Man autônomo).
- Fantasmas como agentes baseados em objetivos que perseguem o Pac-Man recalculando, a cada passo, o caminho de menor custo com A* (heurística de distância de Manhattan) — replanejamento contínuo em ambiente que muda enquanto o agente delibera.
- **Personalidades de fantasmas:** perseguidor direto (mira a posição atual), emboscador (mira a

posição futura prevista) e cauteloso (recua ao se aproximar) — todos sobre o mesmo motor A*, apenas com alvos diferentes.

- **Inversão de comportamento por evento:** ao coletar uma pílula de poder, os fantasmas alternam de

"caçar" para "fugir" do Pac-Man, evidenciando agentes que percebem e reagem a mudanças de estado do

ambiente.

- Pac-Man autônomo que coleta pílulas e foge usando Greedy Best-First, permitindo comparar

A* × Greedy ao vivo.

- **Classificação Naive Bayes** do nível de ameaça em tempo real, com painel mostrando as probabilidades de cada classe (seguro / risco / crítico) e a postura resultante do Pac-Man.

- **Painel de métricas em tempo real** (nós expandidos, comprimento do caminho e tempo de cálculo por

agente) para evidenciar o impacto das heurísticas.

- **Controles de simulação:** alternância de modo, play / pause / passo a passo e ajuste de velocidade.

Tecnologias e técnicas de IA

Stack: TypeScript sobre Next.js 15 e React 19, com renderização do jogo em Canvas

e interface estilizada com Tailwind CSS v4. Testes com Vitest. Aplicação 100% client-side,

publicada como _static export_ no GitHub Pages via GitHub Actions.

Arquitetura: o motor de jogo (`lib/game/`) é TypeScript puro, sem dependência de React/DOM — testável e portátil. A camada de apresentação (`app/` , `components/` , `hooks/`) apenas renderiza o estado e envia comandos. O classificador Naive Bayes é treinado **no próprio navegador**, na inicialização, com estados simulados rotulados.

Tipos de agente: agentes baseados em objetivos (goal-based) com modelo interno do ambiente. O ambiente é totalmente observável, multiagente, dinâmico, discreto e sequencial, e essencialmente determinístico.

Heurísticas de busca: A* (fantasmas) e Greedy Best-First (Pac-Man), ambas usando a distância de Manhattan — admissível e consistente em grid com movimento em 4 direções. As duas estratégias compartilham a mesma fronteira best-first; mudam apenas a função de avaliação.

Naive Bayes (Gaussiano): classificação probabilística do nível de ameaça do estado de jogo a partir de atributos numéricos (distância ao fantasma mais próximo, número de fantasmas em rota de colisão, pílulas de poder disponíveis, pílulas restantes), usado para ajustar a postura do Pac-Man

autônomo entre coletar e fugir.

Limitações conhecidas / fora de escopo (v1.0)

- Labirinto estático e pré-definido (sem editor de mapas nem geração procedural).
- Sem aprendizado por reforço entre partidas: a inteligência de navegação vem do replanejamento de

busca a cada passo. O Naive Bayes é treinado uma vez na inicialização, com dados simulados, e não evolui durante a partida.

- Single-player local apenas (sem rede ou multiplayer).
- Pontuação, vidas e progressão de fases entram de forma simplificada; o foco é o comportamento dos

agentes.

Possíveis evoluções (v1.1+)

- Q-Learning para o Pac-Man aprender políticas de fuga e coleta com a experiência.
- Editor de labirintos e geração procedural de mapas.
- Modo competitivo entre IAs (múltiplos Pac-Man ou estratégias rivais).
- Mais heurísticas no comparador (distância euclidiana, BFS / Dijkstra) no mesmo painel de métricas.
- Treino contínuo / persistente do classificador a partir de partidas reais registradas.

FAQ — Perguntas Frequentes

Como cada fantasma decide para onde se mover? A cada passo, o fantasma trata sua célula atual como

nó inicial e a célula-alvo da sua personalidade (em geral a posição do Pac-Man) como objetivo, roda A* sobre o grafo do labirinto e executa apenas o primeiro passo do caminho retornado. Como o alvo se move, o caminho é recalculado continuamente — é esse replanejamento que faz o agente reagir ao ambiente dinâmico.

Quais são as entradas e saídas do sistema? _Entradas:_ o mapa do labirinto (matriz de paredes/caminhos), as posições de Pac-Man e fantasmas, a localização das pílulas e, no modo Jogador,

as teclas de direção. _Saídas:_ o próximo movimento de cada agente, o estado do jogo atualizado e renderizado no Canvas, a classificação de ameaça do Naive Bayes e as métricas de busca (nós expandidos, custo do caminho e tempo de cálculo).

****Por que A* para os fantasmas e Greedy para o Pac-Man? E por que distância de Manhattan?*** A* garante

o caminho de menor custo, ideal para o perseguidor que quer alcançar o Pac-Man de forma ótima. A distância de Manhattan é admissível e consistente num grid de 4 conexões (nunca superestima o custo real), preservando a otimalidade do A* com bom desempenho. O Pac-Man usa Greedy Best-First porque, ao

fugir, decisões rápidas e "boas o suficiente" importam mais que a rota ótima: Greedy expande menos nós

e responde mais rápido, ao custo de às vezes cair em becos — o que torna a comparação entre os dois

didática e visível no painel de métricas.

O sistema aprende ao longo do tempo? A navegação não: toda a inteligência de perseguição e fuga vem

do replanejamento de busca a cada passo. Há, porém, um modelo treinado — o classificador Naive Bayes —

que aprende, na inicialização, a separar estados seguros de estados críticos a partir de exemplos simulados rotulados. Esse modelo não é re-treinado durante a partida. Aprendizado por reforço (Q-Learning) está previsto como evolução.

Onde entra o classificador Naive Bayes? Ele classifica o estado do Pac-Man em níveis de ameaça (seguro / risco / crítico) a partir de atributos como distância ao fantasma mais próximo, número de fantasmas em rota de colisão, pílulas de poder disponíveis e pílulas restantes. O modelo é um Naive Bayes Gaussiano, treinado no navegador com estados rotulados de partidas simuladas, e seu resultado

ajusta a postura do Pac-Man autônomo — priorizar coleta quando "seguro", priorizar fuga quando "crítico". É exatamente uma decisão sob incerteza, o caso de uso natural do classificador. O painel da interface mostra as probabilidades de cada classe ao vivo.

Como os fantasmas têm comportamentos diferentes se usam o mesmo algoritmo? Todos compartilham o

mesmo motor A*, mas recebem alvos diferentes: o perseguidor mira a célula atual do Pac-Man; o emboscador mira algumas células à frente da direção dele (predição simples); o cauteloso persegue, mas

troca o alvo para um ponto de recuo quando a distância fica pequena. Isso gera personalidades distintas

com custo mínimo de código.

O que acontece quando o Pac-Man pega uma pílula de poder? O estado global muda para "modo poder"

por tempo limitado: os fantasmas invertem o objetivo e passam a maximizar a distância ao Pac-Man (fuga,

via Greedy), enquanto o Pac-Man pode capturá-los. Isso demonstra agentes que percebem e reagem a

mudanças no ambiente, alternando estratégia conforme o estado.

O ambiente é determinístico? Como garantir desempenho em tempo real? O ambiente é discreto, totalmente observável e essencialmente determinístico — cada ação leva a um estado previsível.

Como o

labirinto é pequeno (dezenas a poucas centenas de células), uma busca A* por agente a cada passo é

barata, mais a classificação Naive Bayes (que é apenas avaliação de gaussianas, $O(\text{atributos} \times \text{classes})$).

O painel de métricas comprova que o tempo de cálculo permanece na casa de milissegundos.